



RULETA JAIDER

CIBERSEGURIDAD

MANUAL TÉCNICO COMPLETO

Aplicación Web de Sorteo Aleatorio · Firebase · HTML5

Campo	Valor
Aplicación	Ruleta Jaider — Sorteo de Ciberseguridad
Tecnología	HTML5 · CSS3 · JavaScript ES6+ · Firebase
Versión	2.0 — Producción
Proyecto Firebase	ruleta-jaider
URL de producción	https://ruleta-jaider.web.app
Última actualización	12 de marzo de 2026
Autor	Jaider



1. Descripción General de la Aplicación

RULETA JAIDER es una aplicación web de sorteo aleatorio con temática de casino que permite cargar listas de participantes desde archivos Excel (.xlsx), .xls o .csv y elegir ganadores de forma aleatoria mediante una ruleta animada. Está diseñada para uso en el área de Ciberseguridad.

1.1 Características principales

Característica	Descripción
Carga de archivos	Soporte para .xlsx, .xls y .csv. Área de arrastre (drag & drop) o selección manual. Primera columna = lista de nombres.
Ruleta animada casino	Canvas HTML5 con segmentos rojo/negro, aro dorado con puntos decorativos, hub central con ♠, animación easing cúbico.
Exclusión de últimos 7	Los 7 ganadores más recientes quedan excluidos de los próximos 7 sorteos. Se visualizan en el panel derecho.
Efectos sonoros	Web Audio API: ticks de desaceleración realista + arpegio musical ganador (C5-E5-G5-C6).
Confetti	60 símbolos de casino (♠♥♦♣) animados al anunciar ganador.
Múltiples listas	Guardar, cargar, actualizar y eliminar listas independientes. Dropdown desplegable con todas las listas guardadas.
Persistencia Firebase	Firestore almacena listas e historial en la nube. Fallback automático a localStorage si Firebase no está disponible.
Historial completo	Panel inferior con tarjetas de cada sorteo: ganador, lista, fecha/hora. Exportable a Excel.
Estadísticas en tiempo real	Total de giros, participantes, disponibles y excluidos actualizados en cada sorteo.
Diseño responsivo	Grid CSS adaptable a 3 columnas (desktop), 2 (tablet) y 1 (móvil).

1.2 Tecnologías utilizadas

Tecnología	Versión	Uso
Firebase App (compat)	10.7.1	Inicialización del SDK de Firebase
Firebase Firestore (compat)	10.7.1	Base de datos en la nube para listas e historial
Firebase Analytics (compat)	10.7.1	Métricas de uso (measurementId: G-9T2N4TLDLQ)
SheetJS (XLSX)	0.18.5	Parseo de archivos Excel y CSV
Cinzel Decorative	Google Fonts	Tipografía del título y paneles

Tecnología	Versión	Uso
Playfair Display	Google Fonts	Tipografía del subtítulo y ganador
Josefin Sans	Google Fonts	Tipografía base de la interfaz
Canvas 2D API	HTML5 nativo	Dibujo de la ruleta
Web Audio API	HTML5 nativo	Generación de efectos sonoros

2. Estructura del Archivo HTML

La aplicación está contenida en un único archivo HTML auto-suficiente (ruleta-casino.html). No requiere frameworks, bundlers ni herramientas de compilación. Todo el CSS y JavaScript está embebido en el mismo archivo.

2.1 Organización del código

Sección	Líneas aprox.	Contenido
<head> — Dependencias	1–20	SDKs de Firebase (compat), SheetJS, Google Fonts
<style> — CSS	21–850	Variables, layout grid, ruleta, paneles, historial, animaciones, toast, responsive
<body> — HTML	851–870	Banner Firebase, overlay de carga, contenedor de toasts, encabezado, layout principal, sección de historial
<script> — Bloque ①	871–895	firebaseConfig con credenciales reales del proyecto
<script> — Bloque ②	896–905	Estado global: variables de nombres, historial, audio, animación
<script> — Bloque ③	906–910	Inicialización: initFirebase(), drawWheel(), setupDragDrop()
<script> — Bloque ④	911–923	Chips flotantes de fondo (animación CSS)
<script> — Bloque ⑤	924–941	Drag & Drop de archivos (dragover, dragleave, drop, change)
<script> — Bloque ⑥	942–1010	Parseo de Excel/CSV con SheetJS (FileReader, XLSX.read)
<script> — Bloque ⑦	1011–1200	Firebase CRUD de listas: saveAsNewList(), updateCurrentList(), loadLists(), loadSelectedList(), deleteCurrentList()
<script> — Bloque ⑧	1201–1280	Firebase historial: saveSpinResult(), loadHistory(), clearHistory(), exportHistory()
<script> — Bloque ⑨	1281–1380	Lógica del sorteo: pickWinner(), startSpin(), easeOut(), animateSpin(), onSpinComplete(), resetLastSeven()
<script> — Bloque ⑩	1381–1620	Dibujo Canvas: drawWheel(), drawEmptyWheel()
<script> — Bloque ⑪	1621–1699	Actualización UI: updateNamesDisplay(), updateExcludedDisplay(), updateStats(), renderHistory()
<script> — Bloque ⑫	1700–1775	Audio Web API: initAudio(), playTick(), playSpinSound(), stopSpinSound(), playWinSound()
<script> — Bloque ⑬	1776–1795	Confetti: launchConfetti() con 60 elementos animados
<script> — Bloque ⑭	1796–1814	Notificaciones: showToast(), showLoading()

3. Configuración de Firebase

3.1 Datos del proyecto Firebase

Parámetro	Valor configurado
Proyecto ID	ruleta-jaidер
Auth Domain	ruleta-jaidер.firebaseio.com
Storage Bucket	ruleta-jaidер.firebaseiostorage.app
Sender ID	152775477572
App ID	1:152775477572:web:1919488bb591f129d3ef11
Measurement ID (Analytics)	G-9T2N4TLDLQ
URL Hosting	https://ruleta-jaidер.web.app

3.2 Código de configuración en el HTML

Las SDKs se cargan en el <head> antes de cualquier otro script:

```
<!-- Firebase Compat SDK v10 (funciona sin bundler/webpack) -->
<script src="https://www.gstatic.com/firebasejs/10.7.1/firebase-app-compat.js"></script>
<script src="https://www.gstatic.com/firebasejs/10.7.1/firebase-firestore-compat.js"></script>
<script src="https://www.gstatic.com/firebasejs/10.7.1/firebase-analytics-compat.js"></script>
```

La inicialización dentro del bloque <script> principal:

```
const firebaseConfig = {
  apiKey: "AIzaSyDVGuNPxZnUaJMVnK-grGARfqgsSS5eHOs",
  authDomain: "ruleta-jaidер.firebaseio.com",
  projectId: "ruleta-jaidер",
  storageBucket: "ruleta-jaidер.firebaseiostorage.app",
  messagingSenderId: "152775477572",
  appId: "1:152775477572:web:1919488bb591f129d3ef11",
  measurementId: "G-9T2N4TLDLQ"
};

function initFirebase() {
  try {
    firebase.initializeApp(firebaseConfig);
    db = firebase.firestore();
    firebase.analytics(); // Inicializa Analytics
    firebaseReady = true;
    loadLists(); // Carga el dropdown de listas
    loadHistory(); // Carga el historial
  } catch (e) {
    // Activa modo offline con localStorage como fallback
    document.getElementById("firebase-warning").classList.add("show");
    loadLocalFallback();
  }
}
```

```
}
```

3.3 Estructura de datos en Firestore

La base de datos usa dos colecciones principales:

Colección	Campo	Tipo	Descripción
listas	name	string	Nombre de la lista (ej. "Equipo A")
listas	names	array<string>	Array con todos los nombres de participantes
listas	lastSeven	array<string>	Últimos 7 ganadores (excluidos temporalmente)
listas	totalSpins	number	Contador total de giros de esta lista
listas	createdAt	Timestamp	Fecha/hora de creación (serverTimestamp)
listas	updatedAt	Timestamp	Fecha/hora de última modificación
historial	winner	string	Nombre del ganador del sorteo
historial	listId	string	ID del documento de la lista usada
historial	listName	string	Nombre de la lista (desnormalizado)
historial	spinNum	number	Número de giro (contador secuencial)
historial	timestamp	Timestamp	Fecha/hora del sorteo

4. Explicación Detallada del Código JavaScript

4.1 Estado global (Bloque ②)

Variable	Tipo	Descripción
db	Firestore null	Instancia de Firestore. null hasta que Firebase se inicialice.
firebaseReady	boolean	Indica si Firebase está disponible. false activa el modo localStorage.
currentNames	string[]	Array con los nombres del archivo cargado / lista seleccionada.
lastSeven	string[]	Array circular de máximo 7 elementos con los últimos ganadores excluidos.
localHistory	object[]	Historial completo de sorteos (cargado de Firestore o localStorage).
totalSpins	number	Contador de giros de la sesión actual.
currentListId	string null	ID del documento Firestore de la lista activa. null = lista nueva sin guardar.
currentListName	string	Nombre legible de la lista activa.
isSpinning	boolean	Bloquea el botón GIRAR durante la animación.
currentAngle	number	Ángulo actual de la rueda en radianes (persiste entre giros).
audioCtx	AudioContext	Contexto de Web Audio API. Se inicializa en el primer giro.

4.2 Parseo de archivos — processFile() (Bloque ⑥)

La función processFile() usa FileReader de HTML5 para leer el archivo. Detecta el tipo por extensión:

```
function processFile(file) {
  const isCSV = file.name.toLowerCase().endsWith(".csv");
  if (isCSV) {
    reader.readAsText(file, "UTF-8"); // Lee como texto plano
  } else {
    reader.readAsArrayBuffer(file); // Lee como bytes binarios (Excel)
  }
}

// Para Excel: SheetJS procesa el ArrayBuffer
const workbook = XLSX.read(data, { type: "array" });
const sheet = workbook.Sheets[workbook.SheetNames[0]]; // Primera hoja
const rows = XLSX.utils.sheet_to_json(sheet, { header: 1 }); // Array de arrays
names = rows.map(row => String(row[0] || "").trim()).filter(n => n); // Columna A

// BUG FIX CRÍTICO: Resetear el ID al cargar archivo nuevo
```

```
currentListId = null; // ← sin esto, sobrescribía la lista anterior
currentListName = "";
document.getElementById("btnUpdate").disabled = true;
```

4.3 Gestión de listas — Bloque ⑦ (Corrección de sobrescritura)

Este es el bloque más importante de la versión 2.0. Se separó la función "guardar" en dos funciones distintas para eliminar el bug de sobrescritura:

Función	Método Firestore	Comportamiento
saveAsNewList()	.add(data)	SIEMPRE genera un nuevo ID en Firestore. Nunca pisa documentos existentes. Resetea lastSeven y totalSpins.
updateCurrentList()	.update(data)	SOLO modifica el documento cuyo ID está en currentListId. Pide confirmación antes. Conserva lastSeven y totalSpins.
loadSelectedList()	.get()	Lee el documento seleccionado. Activa el botón "Actualizar" y carga nombres, lastSeven y totalSpins en el estado.
deleteCurrentList()	.delete()	Elimina el documento. Resetea currentListId a null y desactiva el botón "Actualizar".

```
// saveAsNewList() - usa .add() = nuevo documento siempre
const ref = await db.collection("listas").add(data);
currentListId = ref.id; // guarda el nuevo ID

// updateCurrentList() - usa .update() = modifica solo ese doc
await db.collection("listas").doc(currentListId).update(data);
```

4.4 Lógica del sorteo — Bloque ⑨

```
function pickWinner() {
  // Filtrar los últimos 7 excluidos
  const available = currentNames.filter(n => !lastSeven.includes(n));
  // Si todos están excluidos, usar lista completa (nunca queda bloqueada)
  const pool = available.length > 0 ? available : currentNames;
  return pool[Math.floor(Math.random() * pool.length)];
}

// Calcular ángulo destino para que el ganador quede bajo el puntero
const winnerIndex = currentNames.indexOf(winner);
const segAngle = (2 * Math.PI) / N;
let targetAngle = -(winnerIndex * segAngle + segAngle / 2);
targetAngle = ((targetAngle % (2*Math.PI)) + 2*Math.PI) % (2*Math.PI);

// Añadir 5-8 vueltas completas para efecto dramático
const extraSpins = 5 + Math.floor(Math.random() * 4);
spinTotalAngle = diff + extraSpins * 2 * Math.PI;
spinDuration = 4500 + Math.random() * 2000; // 4.5s - 6.5s
```



```
// Easing cúbico (desaceleración natural)
function easeOut(t) { return 1 - Math.pow(1 - t, 4); }
```

4.5 Dibujo de la Ruleta Canvas — Bloque ⑩

Elemento visual	Técnica Canvas
Aro dorado exterior	createRadialGradient() de 3 paradas: #8B6914 → #FFD700 → #8B6914
Puntos decorativos del aro	Bucle de N puntos equidistantes con arc() y gradiente radial individual
Segmentos rojo/negro	moveTo(cx,cy) + arc() + closePath() + fill() alternando colores
Segmentos excluidos	Fondo #2a2a2a, texto tachado con opacidad 0.25
Texto de los nombres	save() → translate(cx,cy) → rotate(angulo_centro) → fillText() a la derecha
Truncado de texto largo	Bucle que acorta el string hasta que measureText().width <= maxWidth
Hub central ♠	Gradiente radial FFF8DC → FFD700 → #4a3800 + texto serif ♠

4.6 Sistema de audio — Bloque ⑫

Función	Descripción técnica
initAudio()	Crea AudioContext si no existe. Llama .resume() si está en estado "suspended" (política de autoplay de Chrome).
playTick(volume)	Crea OscillatorNode tipo "square". Baja la frecuencia de 1200Hz a 600Hz en 30ms (sonido de click mecánico).
playSpinSound(duration)	Programa una serie de setTimeout con intervalo creciente (interval *= 1.022) para simular la desaceleración de la pelota.
stopSpinSound()	Cancela todos los setTimeout pendientes con clearTimeout para evitar ticks después del giro.
playWinSound()	Toca un arpegio ascendente: C5 (523Hz), E5 (659Hz), G5 (783Hz), C6 (1046Hz) con 180ms de separación.

5. Código Fuente Completo

El archivo completo **ruleta-casino.html** se incluye a continuación. Total: **1815 líneas** de código. Puedes copiar el contenido completo desde la sección 5.1 hasta el final.

⚠ ARCHIVO LISTO PARA PRODUCCIÓN

El código ya incluye las credenciales reales de Firebase (proyecto: ruleta-jaidler). No es necesario modificar nada. Solo cópialo en la carpeta public/ y despliega con Firebase CLI.

5.1 Head — Dependencias y CSS

Líneas 1–850: DOCTYPE, meta tags, SDKs de Firebase, SheetJS, Google Fonts y todos los estilos CSS.

```
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>🎰 Ruleta Jaider · Ciberseguridad</title>

<!-- =====
      PASO 1: Reemplaza con tu configuración Firebase
    ===== -->

<!-- Firebase Compat SDK v10 -->
<script src="https://www.gstatic.com/firebasejs/10.7.1/firebase-app-compat.js"></script>
<script src="https://www.gstatic.com/firebasejs/10.7.1/firebase-firestore-
compat.js"></script>
<script src="https://www.gstatic.com/firebasejs/10.7.1/firebase-analytics-
compat.js"></script>
<!-- SheetJS para leer archivos Excel/CSV -->
<script
src="https://cdnjs.cloudflare.com/ajax/libs/xlsx/0.18.5/xlsx.full.min.js"></script>

<link rel="preconnect" href="https://fonts.googleapis.com">
<link
href="https://fonts.googleapis.com/css2?family=Cinzel+Decorative:wght@700;900&family=Play
fair+Display:ital,wght@0,400;0,700;1,400&family=Josefin+Sans:wght@300;400;600;700&display
=swap" rel="stylesheet">

<style>
/* =====
      VARIABLES & RESET
    ===== */
:root {
  --gold:          #FFD700;
  --gold-dark:     #B8860B;
  --gold-light:    #FFF8DC;
  --gold-dim:      #8B6914;
  --red:           #C41E3A;
  --red-dark:      #8B0000;
  --black:         #111111;
  --felt:          #0a3d2e;
  --felt-dark:     #062a1f;
  --cream:         #F5E6D3;
  --bg:            #0d0d0d;
```

```

--panel:      rgba(255,255,255,0.04);
--panel-border: rgba(255,215,0,0.2);
--shadow:      0 8px 32px rgba(0,0,0,0.8);
--glow-gold:    0 0 20px rgba(255,215,0,0.4);
--glow-red:     0 0 20px rgba(196,30,58,0.6);
--radius:      16px;
}

*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }

html { scroll-behavior: smooth; }

body {
  background: var(--bg);
  background-image:
    radial-gradient(ellipse at 20% 20%, rgba(139,0,0,0.15) 0%, transparent 50%),
    radial-gradient(ellipse at 80% 80%, rgba(184,134,11,0.1) 0%, transparent 50%);
  min-height: 100vh;
  font-family: 'Josefin Sans', sans-serif;
  color: var(--cream);
  overflow-x: hidden;
}

/* =====
   FLOATING CHIPS BACKGROUND
   ===== */

.bg-chips {
  position: fixed; inset: 0; pointer-events: none; z-index: 0; overflow: hidden;
}

.bg-chip {
  position: absolute;
  width: 40px; height: 40px;
  border-radius: 50%;
  border: 3px solid rgba(255,215,0,0.15);
  opacity: 0.08;
  animation: floatChip linear infinite;
}

@keyframes floatChip {
  from { transform: translateY(110vh) rotate(0deg); }
  to   { transform: translateY(-110vh) rotate(720deg); }
}

/* =====
   FIREBASE WARNING BANNER
   ===== */

#firebase-warning {
  background: linear-gradient(90deg, #7a4f00, #b8860b, #7a4f00);
  color: #fff8dc;
  text-align: center;
  padding: 8px 20px;
  font-size: 0.8rem;
  letter-spacing: 0.05em;
  position: relative; z-index: 10;
  display: none;
}

#firebase-warning.show { display: block; }

/* =====
   HEADER
   ===== */

header {
  position: relative; z-index: 5;

```

```
text-align: center;
padding: 32px 20px 20px;
```

[... continúa hasta línea 850 ...]

5.2 Body — Estructura HTML

Las líneas 851–870 contienen el HTML visible: banner, overlay, header, grid principal e historial.

```
/* =====
  ① CONFIGURACIÓN FIREBASE
  Reemplaza estos valores con los de tu
  proyecto en Firebase Console:
  https://console.firebase.google.com
  ===== */

const firebaseConfig = {
  apiKey: "AIzaSyDVGZnUaJMVnK-grGARfqqgSS5eHOs",
  authDomain: "ruleta-jaider.firebaseio.com",
  projectId: "ruleta-jaider",
  storageBucket: "ruleta-jaider.firebaseio.com",
  messagingSenderId: "152775477572",
  appId: "1:152775477572:web:1988bb591f129d3ef11",
  measurementId: "G-9T2N4TLDLQ"
};

/* =====
  ② ESTADO GLOBAL DE LA APLICACIÓN
  ===== */

let db = null; // Instancia Firestore
let firebaseReady = false; // ¿Firestore listo?

let currentNames = []; // Nombres de la lista activa
let lastSeven = []; // Últimos 7 ganadores (excluidos)
let localHistory = []; // Historial completo
let totalSpins = 0; // Contador de giros

let currentListId = null; // ID del documento en Firestore
let currentListName = ''; // Nombre de la lista seleccionada

let isSpinning = false;
let currentAngle = 0; // Ángulo actual de la rueda (radianes)
let pendingWinner = null; // Ganador calculado antes de girar

// Animación
let spinStartTime = null;
let spinDuration = 0;
let spinTotalAngle = 0;
let spinStartAngle = 0;
let rafId = null;

// Audio
let audioCtx = null;
let tickIds = []; // IDs de setTimeout de ticks

/* =====
  ③ INICIALIZACIÓN
  ===== */

window.addEventListener('load', () => {
  initBackgroundChips();
```

5.3 JavaScript — Bloques ① a ⑦ (Firebase y Listas)

```

let isSpinning    = false;
let currentAngle  = 0;           // Ángulo actual de la rueda (radianes)
let pendingWinner = null;        // Ganador calculado antes de girar

// Animación
let spinStartTime = null;
let spinDuration  = 0;
let spinTotalAngle = 0;
let spinStartAngle = 0;
let rafId         = null;

// Audio
let audioCtx      = null;
let tickIds       = [];          // IDs de setTimeout de ticks

/* =====
   ③ INICIALIZACIÓN
   ===== */
window.addEventListener('load', () => {
  initBackgroundChips();
  initFirebase();
  drawWheel(0);
  setupDragDrop();
});

function initFirebase() {
  try {
    firebase.initializeApp(firebaseConfig);
    db = firebase.firestore();
    // Inicializar Analytics (opcional, para estadísticas de uso)
    firebase.analytics();
    firebaseReady = true;
    loadLists();
    loadHistory();
    console.log("✅ Firebase conectado - proyecto: ruleta-jaider");
  } catch (e) {
    console.warn("⚠️ Firebase no disponible:", e.message);
    document.getElementById('firebase-warning').classList.add('show');
    loadLocalFallback();
  }
}

/* =====
   ④ FONDO ANIMADO
   ===== */
function initBackgroundChips() {
  const container = document.getElementById('bgChips');
  const colors = ['#C41E3A', '#B8860B', '#111111', '#1a3a1a'];
  for (let i = 0; i < 18; i++) {
    const chip = document.createElement('div');
    chip.className = 'bg-chip';
    chip.style.left = Math.random() * 100 + '%';
    chip.style.borderColor = colors[i % colors.length];
    chip.style.animationDuration = (15 + Math.random() * 20) + 's';
    chip.style.animationDelay = (-Math.random() * 20) + 's';
    chip.style.width = chip.style.height = (20 + Math.random() * 30) + 'px';
  }
}

```

```

    container.appendChild(chip);
  }
}

/* =====
   ⑤ DRAG & DROP ARCHIVO
===== */

function setupDragDrop() {
  const zone = document.getElementById('dropZone');
  zone.addEventListener('dragover', e => { e.preventDefault(); zone.classList.add('drag-over'); });
  zone.addEventListener('dragleave', () => zone.classList.remove('drag-over'));
  zone.addEventListener('drop', e => {
    e.preventDefault();
    zone.classList.remove('drag-over');
    const file = e.dataTransfer.files[0];
    if (file) processFile(file);
  });
  document.getElementById('fileInput').addEventListener('change', e => {
    if (e.target.files[0]) processFile(e.target.files[0]);
  });
}

/* =====
   ⑥ PARSEO DE ARCHIVO EXCEL / CSV
===== */

function processFile(file) {
  showLoading(true);
  const reader = new FileReader();
  const isCSV = file.name.toLowerCase().endsWith('.csv');

  reader.onload = function(e) {
    try {
      let names = [];

      if (isCSV) {
        // Parseo CSV simple
        const text = e.target.result;
        const lines = text.split(/\r?\n/);
        names = lines
          .map(l => l.split(',')[0].trim().replace(/^['"]|['"]$/g, ''))
          .filter(n => n && n.length > 0);
      } else {
        // Parseo Excel con SheetJS
        const data = new Uint8Array(e.target.result);
        const workbook = XLSX.read(data, { type: 'array' });
        const sheet = workbook.Sheets[workbook.SheetNames[0]];
        const rows = XLSX.utils.sheet_to_json(sheet, { header: 1 });
        names = rows
          .map(row => String(row[0] || '').trim())
          .filter(n => n && n.length > 0);
      }

      // Eliminar encabezados numéricos accidentales
      if (names.length > 0 && !isNaN(names[0]) === false &&
names[0].toLowerCase().includes('nombre')) {
        names.shift();
      }

      if (names.length === 0) {
        showToast('No se encontraron nombres en el archivo', 'error');
      }
    }
  };
}

```

```

        showLoading(false);
        return;
    }

    currentNames = names;
    // — BUG FIX: al cargar un archivo nuevo siempre es una lista nueva —
    currentListId = null;
    currentListName = '';
    document.getElementById('btnUpdate').disabled = true;
    // =====
    document.getElementById('fileBadge').classList.add('show');
    document.getElementById('fileName').textContent = file.name;
    document.getElementById('fileCount').textContent = `${names.length} nombre(s)
encontrado(s)`;
    document.getElementById('listNameInput').value = file.name.replace(/\.([^.]+\$/, ' ');

    updateNamesDisplay();
    drawWheel(currentAngle);
    updateStats();
    showToast(`☑ ${names.length} nombres cargados`, 'success');

    } catch (err) {
        showToast('Error al leer el archivo: ' + err.message, 'error');
        console.error(err);
    }
    showLoading(false);
};

reader.onerror = () => { showToast('Error al leer el archivo', 'error');
showLoading(false); };

if (isCSV) {
    reader.readAsText(file, 'UTF-8');
} else {
    reader.readAsArrayBuffer(file);
}
}

/* =====
⑦ FIREBASE — LISTAS
===== */

/** Siempre crea un documento NUEVO en Firestore (nunca sobrescribe) */
async function saveAsNewList() {
    const listName = document.getElementById('listNameInput').value.trim();
    if (!listName) { showToast('Escribe un nombre para la lista', 'error'); return; }
    if (currentNames.length === 0) { showToast('Carga un archivo primero', 'error');
return; }

    const data = {
        name: listName,
        names: currentNames,
        lastSeven: [], // nueva lista = sin exclusiones
        totalSpins: 0,
        createdAt: firebaseReady ? firebase.firestore.FieldValue.serverTimestamp() : new
Date().toISOString(),
        updatedAt: firebaseReady ? firebase.firestore.FieldValue.serverTimestamp() : new
Date().toISOString()
    };

    showLoading(true);

```

```

    try {
      if (firebaseReady) {
        const ref = await db.collection('listas').add(data);    // ← .add() = nuevo ID
siempre
        currentListId    = ref.id;
        currentListName = listName;
        lastSeven        = [];
        totalSpins       = 0;
        document.getElementById('btnUpdate').disabled = false;
        await loadLists();
        document.getElementById('listsDropdown').value = currentListId;
        showToast(`☑ Lista "${listName}" creada`, 'success');
      } else {
        const key = 'lista_' + Date.now();                      // ← clave única con
timestamp
        const all = JSON.parse(localStorage.getItem('ruleta_listas') || '{}');
        all[key]      = data;
        currentListId = key;
        currentListName = listName;
        lastSeven      = [];
        totalSpins     = 0;
        localStorage.setItem('ruleta_listas', JSON.stringify(all));
        document.getElementById('btnUpdate').disabled = false;
        loadLocalFallback();
        document.getElementById('listsDropdown').value = key;
        showToast(`☑ Lista "${listName}" creada localmente`, 'success');
      }
      updateStats();
      updateExcludedDisplay();
    } catch (err) {
      showToast('Error al guardar: ' + err.message, 'error');
    }
    showLoading(false);
  }

  /** Sobreescribe SOLO la lista que está seleccionada actualmente */
  async function updateCurrentList() {
    if (!currentListId) { showToast('Selecciona primero una lista del dropdown', 'error');
return; }
    const listName = document.getElementById('listNameInput').value.trim();
    if (!listName) { showToast('Escribe un nombre para la lista', 'error'); return; }
    if (currentNames.length === 0) { showToast('No hay nombres en la lista', 'error');
return; }

    if (!confirm(`¿Actualizar la lista "${currentListName}" con los datos actuales?`))
return;

    const data = {
      name:      listName,
      names:     currentNames,
      lastSeven: lastSeven,
      totalSpins: totalSpins,
      updatedAt: firebaseReady ? firebase.firestore.FieldValue.serverTimestamp() : new
Date().toISOString()
    };

    showLoading(true);
    try {
      if (firebaseReady) {
        await db.collection('listas').doc(currentListId).update(data);    // ← .update() =
solo ese doc
        currentListName = listName;

```



```

    await loadLists();
    document.getElementById('listsDropdown').value = currentListId;
    showToast(`☑ Lista "${listName}" actualizada`, 'success');
  } else {

```

5.4 JavaScript — Bloques ⑧ a ⑫ (Sorteo, Canvas, Audio)

```

    const all = JSON.parse(localStorage.getItem('ruleta_listas') || '{}');
    all[currentListId] = { ...all[currentListId], ...data };
    localStorage.setItem('ruleta_listas', JSON.stringify(all));
    loadLocalFallback();
    showToast(`☑ Lista actualizada localmente`, 'success');
  }
} catch (err) {
  showToast('Error al actualizar: ' + err.message, 'error');
}
showLoading(false);
}

async function loadLists() {
  try {
    const dropdown = document.getElementById('listsDropdown');
    const prev = dropdown.value;

    if (firebaseReady) {
      const snap = await db.collection('listas').orderBy('updatedAt', 'desc').get();
      dropdown.innerHTML = '<option value="">- Seleccionar lista -</option>';
      snap.forEach(doc => {
        const opt = document.createElement('option');
        opt.value = doc.id;
        opt.textContent = doc.data().name + ' (' + doc.data().names.length + ')';
        dropdown.appendChild(opt);
      });
    }

    if (prev) dropdown.value = prev;
  } catch (err) {
    console.error('Error cargando listas:', err);
  }
}

async function loadSelectedList() {
  const id = document.getElementById('listsDropdown').value;
  if (!id) return;

  showLoading(true);
  try {
    let data;
    if (firebaseReady) {
      const doc = await db.collection('listas').doc(id).get();
      if (!doc.exists) { showToast('Lista no encontrada', 'error'); showLoading(false);
return; }
      data = doc.data();
    } else {
      const all = JSON.parse(localStorage.getItem('ruleta_listas') || '{}');
      data = all[id];
    }

    currentListId = id;

```

```

    currentListName = data.name;
    currentNames    = data.names || [];
    lastSeven       = data.lastSeven || [];
    totalSpins      = data.totalSpins || 0;

    document.getElementById('btnUpdate').disabled = false; // habilitar Actualizar
    document.getElementById('listNameInput').value = data.name;
    document.getElementById('fileBadge').classList.add('show');
    document.getElementById('fileName').textContent = data.name;
    document.getElementById('fileCount').textContent = `${currentNames.length}
participante(s)`;

    updateNamesDisplay();
    updateExcludedDisplay();
    updateStats();
    drawWheel(currentAngle);
    showToast(`☑ Lista "${data.name}" cargada`, 'success');
  } catch (err) {
    showToast('Error al cargar lista: ' + err.message, 'error');
  }
  showLoading(false);
}

async function deleteCurrentList() {
  const id = document.getElementById('listsDropdown').value;
  if (!id) { showToast('Selecciona una lista primero', 'error'); return; }
  if (!confirm('¿Eliminar esta lista permanentemente?')) return;

  try {
    if (firebaseReady) {
      await db.collection('listas').doc(id).delete();
    } else {
      const all = JSON.parse(localStorage.getItem('ruleta_listas') || '{}');
      delete all[id];
      localStorage.setItem('ruleta_listas', JSON.stringify(all));
    }
    currentListId = null;
    currentNames  = [];
    lastSeven     = [];
    document.getElementById('listNameInput').value = '';
    document.getElementById('btnUpdate').disabled = true;
    document.getElementById('fileBadge').classList.remove('show');
    updateNamesDisplay();
    updateExcludedDisplay();
    updateStats();
    drawWheel(currentAngle);
    await loadLists();
    showToast('Lista eliminada', 'info');
  } catch (err) {
    showToast('Error al eliminar: ' + err.message, 'error');
  }
}

function refreshLists() {
  if (firebaseReady) loadLists();
  else loadLocalFallback();
}

function loadLocalFallback() {
  const all = JSON.parse(localStorage.getItem('ruleta_listas') || '{}');
  const dropdown = document.getElementById('listsDropdown');

```

```

const prev = dropdown.value;
dropdown.innerHTML = '<option value="">- Seleccionar lista -</option>';
Object.entries(all).forEach(([id, data]) => {
  const opt = document.createElement('option');
  opt.value = id;
  opt.textContent = data.name + ' (' + (data.names||[]).length + ')';
  dropdown.appendChild(opt);
});
if (prev) dropdown.value = prev;

// Cargar historial local
const hist = JSON.parse(localStorage.getItem('ruleta_history') || '[]');
localHistory = hist;
renderHistory();
}

/* =====
  ⑧ FIREBASE - HISTORIAL
  ===== */
async function saveSpinResult(winner) {
  const entry = {
    winner: winner,
    listId: currentListId || 'local',
    listName: currentListName || 'Sin nombre',
    spinNum: totalSpins,
    timestamp: firebaseReady
      ? firebase.firestore.FieldValue.serverTimestamp()
      : new Date().toISOString()
  };

  localHistory.unshift({ ...entry, timestamp: new Date().toISOString() });

  if (firebaseReady) {
    try {
      await db.collection('historial').add(entry);
      // Actualizar el lastSeven y totalSpins en la lista
      if (currentListId) {
        await db.collection('listas').doc(currentListId).update({
          lastSeven: lastSeven,
          totalSpins: totalSpins,
          updatedAt: firebase.firestore.FieldValue.serverTimestamp()
        });
      }
    } catch (err) {
      console.error('Error guardando en Firebase:', err);
    }
  } else {
    localStorage.setItem('ruleta_history', JSON.stringify(localHistory.slice(0, 200)));
    // Actualizar lista local
    const all = JSON.parse(localStorage.getItem('ruleta_listas') || '{}');
    if (currentListId && all[currentListId]) {
      all[currentListId].lastSeven = lastSeven;
      all[currentListId].totalSpins = totalSpins;
      localStorage.setItem('ruleta_listas', JSON.stringify(all));
    }
  }
}

async function loadHistory() {
  if (!firebaseReady) return;
  try {
    const snap = await db.collection('historial')

```

```

        .orderBy('timestamp', 'desc')
        .limit(100)
        .get();
    localHistory = snap.docs.map(d => ({ id: d.id, ...d.data() }));
    renderHistory();
} catch (err) {
    console.error('Error cargando historial:', err);
}
}

async function clearHistory() {
    if (!confirm('¿Borrar todo el historial de sorteos?')) return;
    localHistory = [];
    if (firebaseReady) {
        try {
            const snap = await db.collection('historial').limit(500).get();
            const batch = db.batch();
            snap.docs.forEach(d => batch.delete(d.ref));
            await batch.commit();
        } catch (err) {
            console.error(err);
        }
    } else {
        localStorage.removeItem('ruleta_history');
    }
    totalSpins = 0;
    updateStats();
    renderHistory();
    showToast('Historial borrado', 'info');
}

function exportHistory() {
    if (localHistory.length === 0) { showToast('No hay historial que exportar', 'error');
    return; }
    const rows = [['#', 'Ganador', 'Lista', 'Fecha']];
    localHistory.forEach((h, i) => {
        const date = h.timestamp instanceof Object
            ? (h.timestamp.toDate ? h.timestamp.toDate().toLocaleString('es-CO') : '-')
            : new Date(h.timestamp).toLocaleString('es-CO');
        rows.push([i+1, h.winner, h.listName, date]);
    });
    const ws = XLSX.utils.aoa_to_sheet(rows);
    const wb = XLSX.utils.book_new();
    XLSX.utils.book_append_sheet(wb, ws, 'Historial');
    XLSX.writeFile(wb, 'historial_ruleta.xlsx');
    showToast('📄 Historial exportado', 'success');
}

/* =====
  ⑨ LÓGICA DEL SORTEO
  ===== */

/**
 * Elige un ganador al azar excluyendo los últimos 7.
 * Si todos están excluidos, permite elegir de la lista completa.
 */
function pickWinner() {
    if (currentNames.length === 0) return null;
    const available = currentNames.filter(n => !lastSeven.includes(n));
    const pool = available.length > 0 ? available : currentNames;
    return pool[Math.floor(Math.random() * pool.length)];
}

```

```

function startSpin() {
  if (isSpinning) return;
  if (currentNames.length === 0) {
    showToast('Carga una lista de nombres primero', 'error');
    return;
  }

  const winner = pickWinner();
  if (!winner) { showToast('No hay ganadores disponibles', 'error'); return; }

  pendingWinner = winner;
  const winnerIndex = currentNames.indexOf(winner);
  const N = currentNames.length;
  const segAngle = (2 * Math.PI) / N;

  // Calcular ángulo objetivo para que el ganador quede en el puntero (arriba)
  // En drawWheel: segmento i empieza en angle + i*segAngle - PI/2
  // Queremos que el centro del segmento ganador esté en PI/2 (arriba del canvas = -PI/2
  // en el sistema canvas)
  // angle + winnerIndex * segAngle + segAngle/2 ≡ 0 (mod 2π) [porque restamos PI/2 en
  // draw y sumamos PI/2 aquí]
  let targetAngle = -(winnerIndex * segAngle + segAngle/2);
  // Normalizar a [0, 2π)
  targetAngle = ((targetAngle % (2 * Math.PI)) + 2 * Math.PI) % (2 * Math.PI);

  const currentNorm = ((currentAngle % (2 * Math.PI)) + 2 * Math.PI) % (2 * Math.PI);
  let diff = targetAngle - currentNorm;
  if (diff < 0) diff += 2 * Math.PI;

  // Añadir 5-8 vueltas completas extra para efecto dramático
  const extraSpins = 5 + Math.floor(Math.random() * 4);
  const totalSpin = diff + extraSpins * 2 * Math.PI;

  spinStartAngle = currentAngle;
  spinTotalAngle = totalSpin;
  spinDuration = 4500 + Math.random() * 2000; // 4.5s - 6.5s
  spinStartTime = null;
  isSpinning = true;

  document.getElementById('spinBtn').disabled = true;
  document.getElementById('winnerDisplay').classList.remove('show');
  document.getElementById('wheelCanvas').classList.add('spinning');

  initAudio();
  playSpinSound(spinDuration);

  requestAnimationFrame(animateSpin);
}

```

5.5 JavaScript — Bloques ⑬ y ⑭ (Confetti y Notificaciones)

```

/* Easing cubic ease-out */
function easeOut(t) {
  return 1 - Math.pow(1 - t, 4);
}

function animateSpin(timestamp) {

```

```

    if (!spinStartTime) spinStartTime = timestamp;
    const elapsed = timestamp - spinStartTime;
    const t      = Math.min(elapsed / spinDuration, 1);
    const eased  = easeOut(t);

    currentAngle = spinStartAngle + spinTotalAngle * eased;
    drawWheel(currentAngle);

    if (t < 1) {
      rafId = requestAnimationFrame(animateSpin);
    } else {
      currentAngle = spinStartAngle + spinTotalAngle;
      drawWheel(currentAngle);
      isSpinning = false;
      onSpinComplete();
    }
  }
}

function onSpinComplete() {
  stopSpinSound();
  document.getElementById('wheelCanvas').classList.remove('spinning');
  document.getElementById('wheelCanvas').classList.add('winner-flash');
  setTimeout(() => document.getElementById('wheelCanvas').classList.remove('winner-
flash'), 2000);

  // Registrar ganador
  totalSpins++;
  lastSeven.unshift(pendingWinner);
  if (lastSeven.length > 7) lastSeven.pop();

  // Mostrar ganador
  document.getElementById('winnerName').textContent = pendingWinner;
  document.getElementById('winnerDisplay').classList.add('show');

  // Actualizar UI
  updateStats();
  updateNamesDisplay();
  updateExcludedDisplay();
  renderHistory();

  // Guardar en Firebase / localStorage
  saveSpinResult(pendingWinner).catch(console.error);

  // Efectos
  playWinSound();
  launchConfetti();

  // Re-habilitar botón
  setTimeout(() => {
    document.getElementById('spinBtn').disabled = false;
  }, 1000);
}

async function resetLastSeven() {
  if (!confirm('¿Limpiar las exclusiones de los últimos 7?')) return;
  lastSeven = [];
  if (firebaseReady && currentListId) {
    try {
      await db.collection('listas').doc(currentListId).update({ lastSeven: [] });
    } catch (e) { console.error(e); }
  }
}

```

```

updateNamesDisplay();
updateExcludedDisplay();
updateStats();
drawWheel(currentAngle);
showToast('Exclusiones limpiadas', 'info');
}

/* =====
   ⑩ DIBUJO DE LA RULETA
   ===== */
function drawWheel(angle) {
  const canvas = document.getElementById('wheelCanvas');
  const ctx = canvas.getContext('2d');
  const W = canvas.width, H = canvas.height;
  const cx = W / 2, cy = H / 2;
  const outerR = Math.min(W, H) / 2 - 22;
  const innerR = outerR * 0.16;

  ctx.clearRect(0, 0, W, H);

  if (currentNames.length === 0) {
    drawEmptyWheel(ctx, cx, cy, outerR, innerR, angle);
    return;
  }

  const N = currentNames.length;
  const segAngle = (2 * Math.PI) / N;

  // — Aro exterior dorado con puntos decorativos —
  const rimGrad = ctx.createRadialGradient(cx, cy, outerR + 4, cx, cy, outerR + 20);
  rimGrad.addColorStop(0, '#8B6914');
  rimGrad.addColorStop(0.5, '#FFD700');
  rimGrad.addColorStop(1, '#8B6914');
  ctx.beginPath();
  ctx.arc(cx, cy, outerR + 18, 0, 2 * Math.PI);
  ctx.fillStyle = rimGrad;
  ctx.fill();

  // Puntos en el aro
  const numDots = Math.max(16, N * 2);
  for (let i = 0; i < numDots; i++) {
    const a = (i / numDots) * 2 * Math.PI;
    const bx = cx + (outerR + 9) * Math.cos(a);
    const by = cy + (outerR + 9) * Math.sin(a);
    const dotGrad = ctx.createRadialGradient(bx - 1, by - 1, 0.5, bx, by, 3.5);
    dotGrad.addColorStop(0, '#FFF8DC');
    dotGrad.addColorStop(1, '#8B6914');
    ctx.beginPath();
    ctx.arc(bx, by, 3.5, 0, 2 * Math.PI);
    ctx.fillStyle = dotGrad;
    ctx.fill();
  }

  // — Segmentos —
  const segColors = [
    { fill: '#B22222', stroke: '#8B0000', text: '#FFE4B5' }, // rojo
    { fill: '#1a1a1a', stroke: '#0d0d0d', text: '#FFD700' } // negro
  ];

  for (let i = 0; i < N; i++) {
    const startA = angle + i * segAngle - Math.PI / 2;

```

```

const endA    = startA + segAngle;
const isExcluded = lastSeven.includes(currentNames[i]);

let colors;
if (isExcluded) {
  colors = { fill: '#2a2a2a', stroke: '#1a1a1a', text: 'rgba(245,230,211,0.25)' };
} else {
  colors = segColors[i % 2];
}

// Sombra interior
ctx.beginPath();
ctx.moveTo(cx, cy);
ctx.arc(cx, cy, outerR, startA, endA);
ctx.closePath();
ctx.fillStyle = colors.fill;
ctx.fill();

// Borde dorado del segmento
ctx.beginPath();
ctx.moveTo(cx, cy);
ctx.arc(cx, cy, outerR, startA, endA);
ctx.closePath();
ctx.strokeStyle = '#FFD700';
ctx.lineWidth = N > 30 ? 0.5 : 1;
ctx.stroke();

// — Texto del nombre —
ctx.save();
ctx.translate(cx, cy);
ctx.rotate(startA + segAngle / 2);

const maxFontSize = 14;
const minFontSize = 7;
const fontSize    = Math.max(minFontSize, Math.min(maxFontSize, 300 / N));
ctx.font = `${isExcluded ? ' ' : 'bold '}${fontSize}px 'Josefin Sans', sans-serif`;
ctx.fillStyle = colors.text;
ctx.textAlign = 'right';
ctx.textBaseline = 'middle';

const maxWidth = outerR - innerR - 12;
let name = currentNames[i];
// Truncar si es demasiado largo
while (ctx.measureText(name).width > maxWidth && name.length > 3) {
  name = name.slice(0, -2) + '...';
}
ctx.fillText(name, outerR - 6, 0);
ctx.restore();
}

// — Círculo interior (hub) —
const hubGrad = ctx.createRadialGradient(cx - 4, cy - 4, 2, cx, cy, innerR);
hubGrad.addColorStop(0, '#FFF8DC');
hubGrad.addColorStop(0.4, '#FFD700');
hubGrad.addColorStop(1, '#4a3800');
ctx.beginPath();
ctx.arc(cx, cy, innerR, 0, 2 * Math.PI);
ctx.fillStyle = hubGrad;
ctx.fill();
ctx.strokeStyle = '#B8860B';
ctx.lineWidth = 2;

```



```

    ctx.stroke();

    // ♠ en el centro
    ctx.font = `bold ${innerR * 0.9}px serif`;
    ctx.fillStyle = '#1a1a1a';
    ctx.textAlign = 'center';
    ctx.textBaseline = 'middle';
    ctx.fillText('♠', cx, cy + 1);
}

function drawEmptyWheel(ctx, cx, cy, outerR, innerR, angle) {
    // Aro dorado
    ctx.beginPath();
    ctx.arc(cx, cy, outerR + 18, 0, 2 * Math.PI);
    ctx.fillStyle = '#B8860B';
    ctx.fill();

    ctx.beginPath();
    ctx.arc(cx, cy, outerR, 0, 2 * Math.PI);
    ctx.fillStyle = '#0d1a0d';
    ctx.fill();

    // Rayos decorativos
    for (let i = 0; i < 12; i++) {
        const a = angle + (i / 12) * 2 * Math.PI;
        ctx.beginPath();
        ctx.moveTo(cx, cy);
        ctx.lineTo(cx + outerR * Math.cos(a), cy + outerR * Math.sin(a));
        ctx.strokeStyle = 'rgba(255,215,0,0.1)';
        ctx.lineWidth = 1;
        ctx.stroke();
    }

    // Texto central
    ctx.font = 'bold 18px Josefin Sans, sans-serif';
    ctx.fillStyle = 'rgba(255,215,0,0.4)';
    ctx.textAlign = 'center';
    ctx.textBaseline = 'middle';
    ctx.fillText('Carga una lista', cx, cy - 12);
    ctx.font = '13px Josefin Sans, sans-serif';
    ctx.fillStyle = 'rgba(255,215,0,0.25)';
    ctx.fillText('para comenzar', cx, cy + 12);

    // Hub
    ctx.beginPath();
    ctx.arc(cx, cy, innerR, 0, 2 * Math.PI);
    ctx.fillStyle = '#B8860B';
    ctx.fill();
}

/* =====
   ⑪ ACTUALIZACIÓN DE LA UI
   ===== */

function updateNamesDisplay() {
    const container = document.getElementById('namesList');
    container.innerHTML = '';
    document.getElementById('participantCount').textContent = currentNames.length;

    if (currentNames.length === 0) {
        container.innerHTML = '<div class="names-empty">Carga un archivo Excel para
comenzar</div>';
    }
}

```

```

    return;
  }
  currentNames.forEach((name, i) => {
    const excluded = lastSeven.includes(name);
    const tag = document.createElement('div');
    tag.className = 'name-tag' + (excluded ? ' excluded' : '');
    tag.innerHTML = `<span class="name-tag-num">${i+1}</span><span
style="flex:1">${name}</span>`;
    container.appendChild(tag);
  });
}

function updateExcludedDisplay() {
  const container = document.getElementById('excludedList');
  container.innerHTML = '';
  if (lastSeven.length === 0) {
    container.innerHTML = '<div class="excluded-empty">Ninguno por ahora</div>';
    return;
  }
  lastSeven.forEach((name, i) => {
    const chip = document.createElement('div');
    chip.className = 'excluded-chip';
    chip.innerHTML = `<span class="ex-num">-${i+1}</span><span>${name}</span>`;
    container.appendChild(chip);
  });
}

function updateStats() {
  const available = currentNames.filter(n => !lastSeven.includes(n)).length;
  document.getElementById('statSpins').textContent = totalSpins;
  document.getElementById('statTotal').textContent = currentNames.length;
  document.getElementById('statAvail').textContent = available;
  document.getElementById('statExcl').textContent = Math.min(lastSeven.length,
currentNames.length);
}

function renderHistory() {
  const grid = document.getElementById('historyGrid');
  grid.innerHTML = '';

  if (localHistory.length === 0) {
    grid.innerHTML = '<div class="history-empty">Aún no hay sorteos realizados</div>';
    return;
  }

  const chipStyles = ['red', 'black', 'gold'];

  localHistory.slice(0, 60).forEach((entry, i) => {
    const card = document.createElement('div');
    card.className = 'history-card';
    const chipClass = chipStyles[i % 3];

    let dateStr = '-';
    if (entry.timestamp) {
      try {
        const d = entry.timestamp instanceof Object && entry.timestamp.toDate
          ? entry.timestamp.toDate()
          : new Date(entry.timestamp);
        dateStr = d.toLocaleTimeString('es-CO', { hour:'2-digit', minute:'2-digit' })
          + ' · ' + d.toLocaleDateString('es-CO', { day:'2-digit', month:'short' });
      } catch(e) {}
    }
  });
}

```

```

    }

    card.innerHTML = `
      <div class="history-chip ${chipClass}">#${entry.spinNum || (localHistory.length -
i)}</div>
      <div class="history-info">
        <div class="history-name">${entry.winner}</div>
        <div class="history-meta">${entry.listName} · ${dateStr}</div>
      </div>`;
    grid.appendChild(card);
  });
}

/* =====
  ⑫ AUDIO (Web Audio API)
  ===== */
function initAudio() {
  if (!audioCtx) {
    audioCtx = new (window.AudioContext || window.webkitAudioContext)();
  }
  if (audioCtx.state === 'suspended') audioCtx.resume();
}

function playTick(volume = 0.3) {
  if (!audioCtx) return;
  try {
    const osc = audioCtx.createOscillator();
    const gain = audioCtx.createGain();
    osc.connect(gain);
    gain.connect(audioCtx.destination);
    osc.type = 'square';
    osc.frequency.setValueAtTime(1200, audioCtx.currentTime);
    osc.frequency.exponentialRampToValueAtTime(600, audioCtx.currentTime + 0.03);
    gain.gain.setValueAtTime(volume, audioCtx.currentTime);
    gain.gain.exponentialRampToValueAtTime(0.001, audioCtx.currentTime + 0.04);
    osc.start(audioCtx.currentTime);
    osc.stop(audioCtx.currentTime + 0.04);
  } catch(e) {}
}

function playSpinSound(duration) {
  tickIds.forEach(clearTimeout);
  tickIds = [];

  // Ticks que empiezan rápido y se desaceleran (efecto ruleta)
  let t = 0;
  let interval = 0.04; // 40ms entre ticks al inicio
  const slowFactor = 1.022; // factor de desaceleración
  let count = 0;

  while (t < duration / 1000 - 0.5) {
    const capturedT = t;
    const capturedCount = count;
    tickIds.push(setTimeout(() => {
      const vol = 0.15 + (capturedCount < 20 ? capturedCount / 20 * 0.2 : 0.2);
      playTick(vol);
    }, capturedT * 1000));
    t += interval;
    interval *= slowFactor;
    count++;
  }
}

```

```

function stopSpinSound() {
  tickIds.forEach(clearTimeout);
  tickIds = [];
}

function playWinSound() {
  if (!audioCtx) return;
  const notes = [523.25, 659.25, 783.99, 1046.50]; // C5 E5 G5 C6
  notes.forEach((freq, i) => {
    try {
      const osc = audioCtx.createOscillator();
      const gain = audioCtx.createGain();
      osc.connect(gain);
      gain.connect(audioCtx.destination);
      osc.type = 'sine';
      const t = audioCtx.currentTime + i * 0.18;
      osc.frequency.setValueAtTime(freq, t);
      gain.gain.setValueAtTime(0, t);
      gain.gain.linearRampToValueAtTime(0.35, t + 0.06);
      gain.gain.exponentialRampToValueAtTime(0.001, t + 0.7);
      osc.start(t);
      osc.stop(t + 0.7);
    } catch(e) {}
  });
}

/* =====
  ⑬ CONFETTI
  ===== */

function launchConfetti() {
  const colors = ['#FFD700', '#C41E3A', '#FFF8DC', '#0a3d2e', '#B8860B', '#ffffff'];
  const shapes = ['●', '■', '▲', '♠', '♥', '♦', '♣'];
  for (let i = 0; i < 60; i++) {
    const el = document.createElement('span');
    el.className = 'confetti-piece';
    el.textContent = shapes[Math.floor(Math.random() * shapes.length)];
    el.style.left = Math.random() * 100 + 'vw';
    el.style.fontSize = (8 + Math.random() * 12) + 'px';
    el.style.color = colors[Math.floor(Math.random() * colors.length)];
    el.style.animationDuration = (1.5 + Math.random() * 2.5) + 's';
    el.style.animationDelay = (Math.random() * 0.5) + 's';
    document.body.appendChild(el);
    el.addEventListener('animationend', () => el.remove());
  }
}

/* =====
  ⑭ TOAST NOTIFICATIONS
  ===== */

function showToast(msg, type = 'info') {
  const container = document.getElementById('toast-container');
  const toast = document.createElement('div');
  toast.className = `toast ${type}`;
  toast.textContent = msg;
  container.appendChild(toast);
  setTimeout(() => toast.remove(), 3200);
}

function showLoading(show) {
  document.getElementById('loadingOverlay').classList.toggle('show', show);
}

```

```
</script>  
</body>  
</html>
```

6. Reglas de Seguridad de Firestore

Las reglas de Firestore controlan quién puede leer y escribir en la base de datos. Deben configurarse en la consola de Firebase antes de desplegar la app.

6.1 Reglas para entorno de desarrollo (abiertas)

Permiten lectura y escritura a cualquier usuario. Úsalas solo durante desarrollo y pruebas:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /listas/{docId} {
      allow read, write: if true;
    }
    match /historial/{docId} {
      allow read, write: if true;
    }
  }
}
```

6.2 Cómo aplicar las reglas

1. Ve a <https://console.firebase.google.com>
2. Selecciona el proyecto ruleta-jaidier
3. En el menú izquierdo: Firestore Database → pestaña Reglas
4. Reemplaza el contenido con las reglas de la sección 6.1
5. Clic en Publicar

Seguridad en producción

Las reglas "allow read, write: if true" permiten acceso público total. Para un entorno de producción real considera agregar Firebase Authentication y restringir el acceso solo a usuarios autenticados.

7. Despliegue en Firebase Hosting — Paso a Paso

Firebase Hosting permite publicar la aplicación en internet en menos de 2 minutos. La URL resultante será `https://ruleta-jaidер.web.app`

7.1 Prerrequisitos

Requisito	Cómo verificar	Cómo instalar si no está
Node.js (LTS)	<code>node --version</code> → debe ser v18+	Descargar desde https://nodejs.org (botón LTS)
npm	<code>npm --version</code> → viene con Node.js	Incluido con la instalación de Node.js
Firebase CLI	<code>firebase --version</code> → debe ser v12+	<code>npm install -g firebase-tools</code>
Cuenta Google	Tener acceso a Gmail	Crear cuenta en https://accounts.google.com
Acceso Firebase	Poder acceder a <code>console.firebase.google.com</code>	Usar la cuenta Google del proyecto ruleta-jaidер

7.2 Instalación de Node.js y Firebase CLI

```
# 1. Verificar que Node.js está instalado
node --version
# Debe mostrar algo como: v20.11.0

# 2. Instalar Firebase CLI globalmente
npm install -g firebase-tools

# 3. Verificar instalación
firebase --version
# Debe mostrar algo como: 13.0.0
```

7.3 Autenticarse con Firebase

```
# Abre el navegador y autentica con tu cuenta de Google
firebase login

# Verifica que estás logueado y puedes ver el proyecto
firebase projects:list
# Debe aparecer: ruleta-jaidер
```

7.4 Crear la estructura de carpetas

Crea la estructura de carpetas para el proyecto. La carpeta public/ contendrá los archivos públicos del sitio web.

```
# En Windows (PowerShell o CMD)
mkdir mi-ruleta
cd mi-ruleta
mkdir public

# En Mac / Linux (Terminal)
mkdir mi-ruleta && cd mi-ruleta && mkdir public

# Copiar el archivo HTML descargado como index.html
# Windows:
copy "C:\ruta\a\ruleta-casino.html" public\index.html

# Mac/Linux:
cp ~/Downloads/ruleta-casino.html public/index.html
```

La estructura debe quedar así:

```
mi-ruleta/
├── public/
│   └── index.html ← el archivo ruleta-casino.html
```

7.5 Inicializar Firebase Hosting en la carpeta

```
# Estando dentro de la carpeta mi-ruleta/
firebase init hosting
```

El asistente interactivo hará las siguientes preguntas:

Pregunta del asistente	Tu respuesta
Which project do you want to associate with this directory?	Selecciona "Use an existing project" → ruleta-jaidier
What do you want to use as your public directory?	Escribe: public
Configure as a single-page app (rewrite all urls to /index.html)?	Escribe: N (No)
Set up automatic builds and deploys with GitHub?	Escribe: N (No)
File public/index.html already exists. Overwrite?	IMPORTANTE: Escribe: N (No) — para conservar tu archivo

Después de responder, se crearán automáticamente dos archivos de configuración:

```
mi-ruleta/
├── public/
│   └── index.html
├── .firebaserc ← vincula esta carpeta con el proyecto "ruleta-jaidier"
└── firebase.json ← configuración de hosting
```

Contenido del firebase.json generado:

```
{
  "hosting": {
    "public": "public",
```



```

    "ignore": [
      "firebase.json",
      "**/*.*",
      "**/node_modules/**"
    ]
  }
}

```

7.6 Desplegar la aplicación

```

# Comando de despliegue (solo hosting, no toca Firestore)
firebase deploy --only hosting

# Salida esperada:
=== Deploying to "ruleta-jaider"...
i  deploying hosting
i  hosting[ruleta-jaider]: beginning deploy...
i  hosting[ruleta-jaider]: found 1 files in public
✓  hosting[ruleta-jaider]: file upload complete
i  hosting[ruleta-jaider]: finalizing version...
✓  hosting[ruleta-jaider]: version finalized
i  hosting[ruleta-jaider]: releasing new version...
✓  hosting[ruleta-jaider]: release complete

✓  Deploy complete!

Project Console: https://console.firebase.google.com/project/ruleta-jaider
Hosting URL:      https://ruleta-jaider.web.app

```

☒ ¡Aplicación publicada!

Tu ruleta estará disponible en <https://ruleta-jaider.web.app> inmediatamente después del despliegue. Comparte esta URL con quien quieras.

7.7 Actualizaciones futuras

Cada vez que modifiques el archivo `index.html`, solo ejecuta:

```

cd mi-ruleta
# ... editas public/index.html ...
firebase deploy --only hosting
# Los cambios estarán en vivo en segundos

```

8. Guía de Uso de la Aplicación

8.1 Cargar una lista nueva

6. Prepara un archivo .xlsx, .xls o .csv con los nombres en la primera columna (columna A)
7. Arrastra el archivo a la zona punteada "Cargar Lista" o haz clic para seleccionarlo
8. La app mostrará cuántos nombres encontró y los listará en el panel izquierdo
9. Los nombres excluidos (últimos 7) aparecerán tachados
10. Escribe un nombre descriptivo en el campo de texto (ej. "Grupo Ciberseguridad 2025")
11. Haz clic en **+** Guardar Nueva para guardar en Firebase

8.2 Seleccionar una lista guardada

12. Abre el desplegable "Listas Guardadas" en el panel izquierdo
13. Selecciona la lista deseada de la lista
14. La app cargará automáticamente: nombres, últimos 7 excluidos, contador de giros
15. Presiona el botón circular rojo GIRAR para iniciar el sorteo

8.3 Diferencia entre "Guardar Nueva" y "Actualizar"

Botón	Cuándo usarlo	Resultado
+ Guardar Nueva	Cuando cargues un archivo nuevo o quieras crear una lista adicional con un nombre diferente	Crea un documento nuevo en Firestore. La lista anterior NO se modifica. Ambas listas coexisten en el dropdown.
 Actualizar	Cuando quieras modificar el nombre o los datos de la lista que está seleccionada actualmente	Sobreescribe SOLO el documento seleccionado. Pide confirmación antes de proceder. Conserva el historial de giros.

8.4 Panel de estadísticas (derecha)

Estadística	Significado
Total giros	Número de sorteos realizados con la lista actual (se guarda en Firestore)
En lista	Total de participantes en la lista activa
Disponibles	Participantes que SÍ pueden ser seleccionados en el próximo giro
Excluidos	Cantidad de participantes bloqueados temporalmente (máximo 7)

8.5 Acciones rápidas

Botón	Función
 Limpiar exclusiones	Vacía el array lastSeven para que todos los participantes vuelvan a estar disponibles inmediatamente
 Limpiar historial	Elimina todos los registros de la colección historial en Firestore (pide confirmación)
 Exportar historial	Genera y descarga un archivo historial_ruleta.xlsx con todos los sorteos: número, ganador, lista, fecha
 Actualizar	Recarga el dropdown de listas desde Firestore (útil si otro usuario guardó una lista desde otro navegador)

9. Solución de Problemas Frecuentes

Problema	Causa probable	Solución
Aparece el banner amarillo de advertencia Firebase	Las credenciales no están configuradas o hay un error de red	Verifica que el firebaseConfig en el HTML tenga los valores correctos del proyecto ruleta-jaidier. La app sigue funcionando en modo local.
No se escucha ningún sonido	El navegador bloqueó el AudioContext por política de autoplay	Haz clic en cualquier parte de la página antes de girar. Chrome requiere interacción del usuario para activar audio.
Las listas no aparecen en el dropdown	Firestore no está habilitado o las reglas bloquean la lectura	Ve a Firebase Console → Firestore → verifica que esté activo y que las Reglas permitan read: if true
Error "Missing or insufficient permissions"	Las reglas de Firestore no permiten escritura	Aplica las reglas de la sección 6.1 y haz clic en Publicar en la consola de Firebase
El archivo Excel no carga	Formato no compatible o archivo corrupto	Guarda el Excel como .xlsx (no .xls antiguo). Asegúrate que los nombres estén en la primera columna (A).
La ruleta no se detiene en el nombre correcto	Normal en listas con más de 50 nombres (precisión visual)	El ganador es elegido antes del giro. El puntero siempre apunta al ganador real aunque visualmente difiera ligeramente.
firebase deploy falla con "not in a Firebase app directory"	No ejecutaste firebase init	Asegúrate de estar dentro de la carpeta mi-ruleta/ y ejecuta firebase init hosting primero
firebase login abre una URL pero no redirige	El navegador predeterminado no abre automáticamente	Copia la URL que muestra la terminal y pégala manualmente en el navegador

10. Historial de Cambios — Versión 2.0

Versión	Cambio	Descripción
2.0	 BUG FIX CRÍTICO	Corregida la sobreescritura de listas. La función <code>saveCurrentList()</code> se dividió en <code>saveAsNewList()</code> (usa <code>.add()</code>) y <code>updateCurrentList()</code> (usa <code>.update()</code>). Al cargar un archivo nuevo, <code>currentListId</code> se resetea a null para evitar que el nuevo contenido sobreescriba listas guardadas.
2.0	 Dos botones de guardado	Se reemplazó el único botón "Guardar" por dos botones distintos:  Guardar Nueva (siempre crea) y  Actualizar (solo activo cuando hay lista seleccionada, pide confirmación).
2.0	 Título actualizado	Título principal cambiado de "CASINO RULETA" a "RULETA JAIDER".
2.0	 Subtítulo actualizado	Subtítulo cambiado de "Sorteo Aleatorio de Nombres" a "CIBERSEGURIDAD".
2.0	 Firebase Analytics	Se agregó el SDK <code>firebase-analytics-compat.js</code> y se llama <code>firebase.analytics()</code> al inicializar para registrar métricas de uso.
1.0	 Versión inicial	Ruleta de casino con Canvas, Firebase Firestore, exclusión de últimos 7, efectos sonoros, confetti, historial, exportación a Excel, diseño responsivo.